

Chapitre 05

Gérer les conteneurs avec Podman

Lancement, ports, volumes, copie de fichiers, variables, Containerfile, réseaux et déploiements multi-conteneurs

Version de révision : schémas explicatifs · termes expliqués · points clés · corrigé des questions

Objectif

Apprendre à gérer des conteneurs avec **Podman** et être capable de réaliser des tâches pratiques : lancement de conteneurs, port mapping, volumes, copie de fichiers, variables d'environnement, construction d'images avec Containerfile, réseaux et déploiements multi-conteneurs.

Idée clé

Tout le chapitre répond à quatre questions : **quoi lancer ?** (image, Containerfile) · **comment l'atteindre ?** (ports) · **où garder les données ?** (volumes, `cp`) · **avec qui parle-t-il ?** (réseaux, variables).

Comment lire ce document

Bloc	Signification
En clair	Explication simple d'un terme important, avec une analogie.
À retenir	Les points clés de la section, à connaître par cœur.
Attention	Pièges, erreurs fréquentes et nuances.
Blocs de code	Commandes à tester dans un terminal avec Podman installé.

Sommaire

0. Introduction à Podman	3
1. Construire une image avec un Containerfile	3
2. Lancer un conteneur	5
3. Port Mapping	5
4. Volumes avec -v	6
5. Les principaux types de montages	7
6. Copier des fichiers avec podman cp	10
7. Variables d'environnement	11
8. ARG vs ENV	11
9. Exemple ARG → ENV	12
10. Les réseaux Podman	13
11. Autres commandes essentielles à connaître	15
Ce qu'il faut absolument retenir	16
Glossaire des termes importants	17
Questions de compréhension — avec corrigés	18

0. Introduction à Podman

Podman utilise une **interface de commandes très proche de Docker** :

```
podman run
podman ps
podman exec
podman cp
podman build
podman network
podman volume
```

Dans ce chapitre, on se concentre sur les commandes utiles pour **administrer des applications conteneurisées**.

En clair — Podman vs Docker pour l'utilisateur

Si vous connaissez Docker, vous connaissez déjà Podman : la plupart du temps, il suffit de remplacer `docker` par `podman`. La différence est « sous le capot » : Podman est **daemonless** et fonctionne nativement en **rootless** (voir le chapitre Container Runtime).

Les noms d'images courts (`nginx`, `mariadb`...) sont utilisés dans les exemples par simplicité. Selon la configuration des registries de votre machine, Podman peut vous demander de choisir le registry ; le nom complet (`docker.io/library/nginx`) évite toute ambiguïté.

1. Construire une image avec un Containerfile

Un **Containerfile** décrit **comment construire une image**. Instructions importantes :

Instruction	Rôle	Exemple
FROM	Image de base	FROM docker.io/library/alpine:latest
ARG	Variable disponible pendant le build	ARG APP_VERSION
ENV	Variable d'environnement	ENV APP_VERSION=1.0
RUN	Exécuter une commande pendant le build	RUN apk add --no-cache nginx
COPY	Copier des fichiers dans l'image	COPY index.html /usr/share/nginx/html/
WORKDIR	Définir le répertoire de travail	WORKDIR /app
EXPOSE	Documenter un port utilisé par l'application	EXPOSE 80
CMD	Commande par défaut	CMD ["nginx", "-g", "daemon off;"]
ENTRYPOINT	Programme principal du conteneur	ENTRYPOINT ["nginx"]

EXPOSE ne publie rien

`EXPOSE 80` ne publie **pas** le port sur l'hôte. Pour publier le port, il faut utiliser `-p`, par exemple `-p 8080:80`.

Exemple

```
FROM docker.io/library/alpine:latest

RUN apk add --no-cache nginx

COPY index.html /usr/share/nginx/html/index.html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Construire :

```
podman build -t custom-nginx .
```

Lancer :

```
podman run -d --name custom-nginx -p 8080:80 custom-nginx
```



Une même image peut servir à lancer plusieurs conteneurs.

Figure 1 — Du Containerfile au conteneur : build puis run.

En clair — Image vs conteneur

Une **image** est un **moule** (ou une recette) ; un **conteneur** est le **gâteau** fabriqué avec ce moule. Avec un seul moule, on peut faire autant de gâteaux qu'on veut. L'image est en lecture seule ; chaque conteneur ajoute sa propre petite couche modifiable.

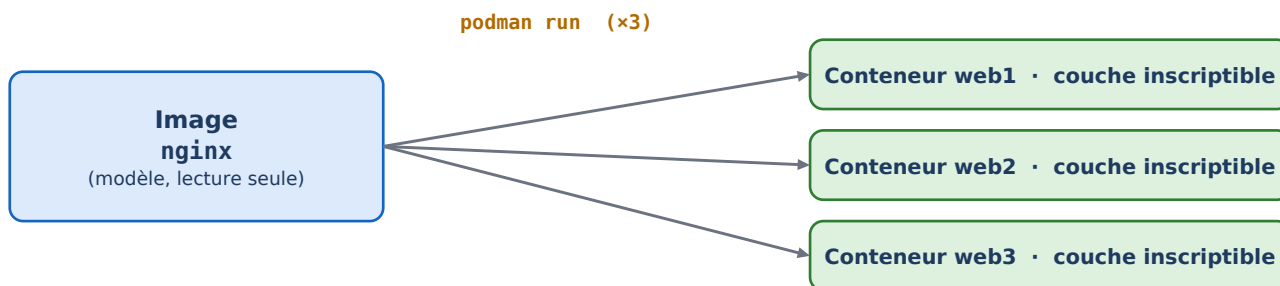


Figure 2 — Une image, plusieurs conteneurs.

Pourquoi « daemon off; » ?

Un conteneur **s'arrête dès que son processus principal se termine**. Par défaut, nginx se met en arrière-plan (« daemon ») et son processus de départ se termine aussitôt : le conteneur s'arrêterait. **daemon off;** force nginx à rester au premier plan pour que le conteneur continue de vivre.

CMD ou ENTRYPOINT ?

CMD définit la commande **par défaut**, facilement remplaçable à l'exécution (`podman run image autre-commande`). **ENTRYPOINT** définit le **programme principal** du conteneur, plus difficile à remplacer (il faut `--entrypoint`).

À retenir

- ✓ Un **Containerfile** = la recette de construction d'une image (`podman build`).
- ✓ **EXPOSE** documente un port, il ne le **publie pas**.
- ✓ Une image = un modèle ; un conteneur = une instance de cette image.

2. Lancer un conteneur

La commande principale est :

```
podman run [OPTIONS] IMAGE
```

Exemple :

```
podman run -d --name web nginx
```

Options importantes

Option	Rôle	Exemple
-d	Exécuter en arrière-plan (<i>detached</i>)	podman run -d nginx
--name	Donner un nom au conteneur	--name web
-p	Publier un port	-p 8080:80
-v	Monter un volume ou un répertoire	-v \$HOME/site:/data:Z
-e	Définir une variable d'environnement	-e RESPONSE="Hello"
--network	Connecter le conteneur à un réseau	--network net
--rm	Supprimer le conteneur à son arrêt	--rm

Vérifier les conteneurs :

```
podman ps
```

Afficher également les conteneurs **arrêtés** :

```
podman ps -a
```

En clair — Mode détaché (-d)

Sans -d, le conteneur occupe votre terminal (premier plan), comme un programme ordinaire. Avec -d, il tourne **en arrière-plan** et le terminal vous est rendu, comme un service qu'on démarre puis qu'on laisse travailler.

À retenir

- ✓ -d arrière-plan · --name nom · -p port · -v volume · -e variable · --network réseau · --rm suppression auto.
- ✓ podman ps = conteneurs en cours ; podman ps -a = tous, y compris arrêtés.

3. Port Mapping

3.1. Principe

Un processus qui écoute dans un conteneur **n'est pas automatiquement accessible** sur un port de l'hôte. On utilise :

```
-p HOST_PORT:CONTAINER_PORT
```

Exemple :

```
podman run -d \
  --name nginx \
  -p 8001:80 \
  nginx
```



Figure 3 — Le port 8001 de l'hôte est redirigé vers le port 80 du conteneur.

On accède alors à Nginx avec :

```
curl http://localhost:8001
```

En clair — Port mapping

Le conteneur est un **bureau avec son propre standard téléphonique** (son réseau isolé). Le port mapping, c'est créer un **renvoi d'appel** : « tout appel arrivant au poste 8001 de l'immeuble (l'hôte) est transféré au poste 80 du bureau (le conteneur) ».

Conseil examen

Toujours lire **HOST:CONTAINER** et **non l'inverse** : à gauche le port de l'hôte, à droite le port du conteneur.

Un port hôte, un seul conteneur

Deux conteneurs ne peuvent pas publier le **même port de l'hôte** en même temps : `-p 8001:80` deux fois provoque une erreur (port déjà utilisé). En revanche, ils peuvent tous les deux écouter sur le port 80 à **l'intérieur** de leur conteneur, car chacun a son propre réseau. Il suffit de les publier sur des ports hôtes différents : `-p 8001:80` et `-p 8002:80`.

À retenir

- ✓ `-p HOTE:CONTENEUR` : le premier nombre est sur la machine, le second dans le conteneur.
- ✓ Sans `-p`, l'application n'est pas joignable depuis l'hôte.

4. Volumes avec -v

Les conteneurs sont **éphémères par nature**. Si les données importantes restent uniquement dans le filesystem du conteneur, elles peuvent être **perdues lorsque le conteneur est supprimé**. Les **volumes** permettent de conserver ou de partager des données **indépendamment du cycle de vie du conteneur**.

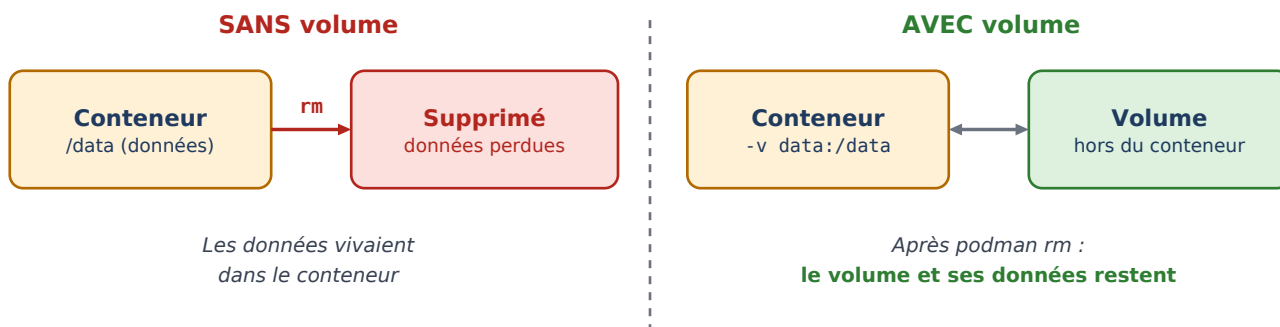


Figure 4 — Sans volume, les données disparaissent avec le conteneur ; avec un volume, elles restent.

La syntaxe générale est :

```
-v SOURCE:DESTINATION
```

Exemple :

```
-v /home/student/web:/usr/share/nginx/html
```

```
Host
/home/student/web
    |
    | mount
    v
Container
/usr/share/nginx/html
```

5. Les principaux types de montages

Avec Podman, on rencontre principalement **trois cas pratiques**.

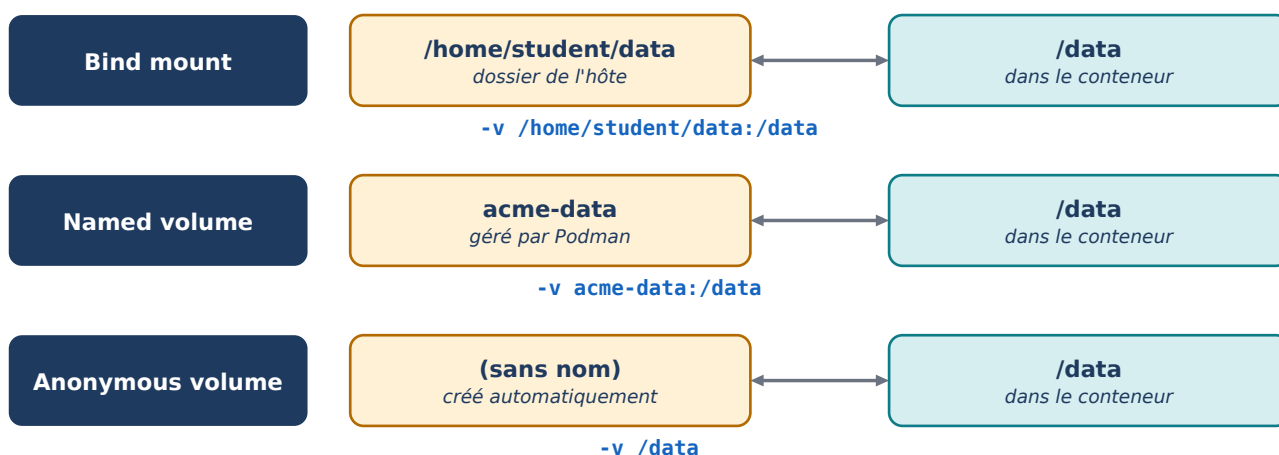


Figure 5 — Les trois types de montage et leur syntaxe.

5.1. Bind mount

On monte un **répertoire ou un fichier existant de l'hôte**. Syntaxe : `-v HOST_PATH:CONTAINER_PATH`.

Exemple :

```
podman run -d \
  --name nginx \
  -v "$HOME/site:/usr/share/nginx/html:Z" \
  nginx
```

```
$HOME/site
  ↓
/usr/share/nginx/html
```

Le contenu du répertoire de l'hôte est **directement accessible** dans le conteneur.

Pourquoi :Z ?

Sur un système avec **SELinux**, le fichier du volume possède un **contexte de sécurité** que le processus du conteneur doit être autorisé à utiliser. Sans `:Z` :

```
-v $HOME/site:/usr/share/nginx/html
```

SELinux peut empêcher Nginx dans le conteneur d'accéder aux fichiers, **même si les permissions Linux classiques** (`chmod`, `chown`) semblent correctes. Avec :

```
-v $HOME/site:/usr/share/nginx/html:Z
```

Podman **relabellise** les fichiers avec un contexte SELinux approprié pour le conteneur, ce qui permet au processus du conteneur d'y accéder.

À retenir

:Z ne sert **pas** à donner des permissions Linux classiques ; il sert à **adapter les permissions de sécurité SELinux** pour que le conteneur puisse accéder au volume.

En clair — SELinux et relabel

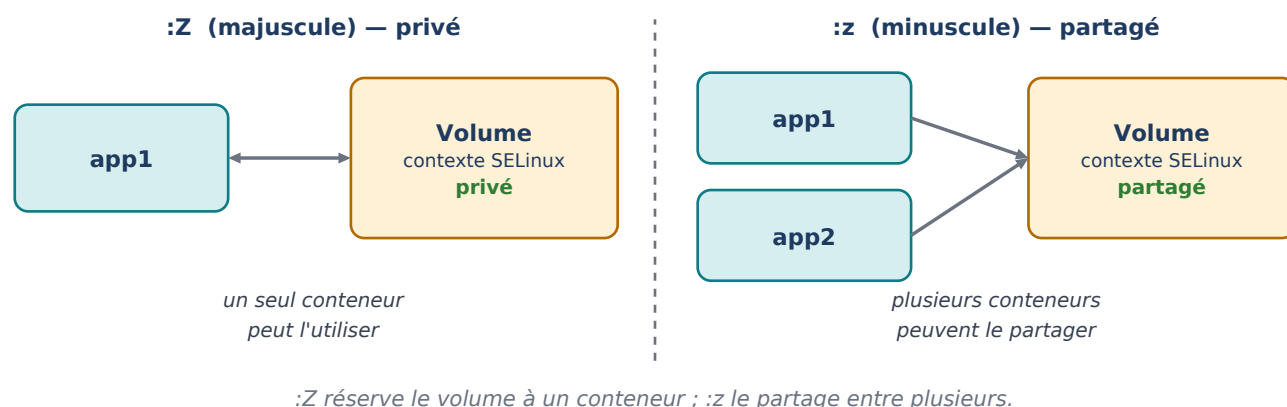
SELinux est un **second système de contrôle d'accès** : même si la porte est déverrouillée par les permissions classiques (`chmod`), le **vigile SELinux** exige un badge portant la bonne étiquette. **Relabelliser**, c'est coller sur le dossier l'étiquette que le vigile accepte pour ce conteneur.

Différence entre :Z et :z

- **:Z** : attribue au volume un contexte SELinux **privé**, destiné à être utilisé par **un seul conteneur**.
- **:z** : attribue au volume un contexte SELinux **partagé**, permettant à **plusieurs conteneurs** d'utiliser le même contenu.

Par exemple :

```
podman run -d --name app1 -v "$HOME/shared-data:/data:z" nginx
podman run -d --name app2 -v "$HOME/shared-data:/data:z" nginx
```



Quand utiliser ce montage ?

- développement ;
- configuration ;
- partage de fichiers ;
- contenu web ;
- scripts ;
- données que l'administrateur souhaite gérer directement depuis l'hôte.

5.2. Named volume

Podman peut créer un **volume géré par Podman**. Créer le volume :

```
podman volume create host-data
```

Vérifier :

```
podman volume ls
```

Monter le volume :

```
podman run -d --name app -v host-data:/data nginx
```

```
host-data
|
▼
/data
```

Le volume est identifié par son **nom**. Le stockage physique est **géré par Podman**.

Quand l'utiliser ?

Les named volumes sont particulièrement pratiques pour : **bases de données, données persistantes, applications qui doivent conserver leurs données, partage de données entre plusieurs conteneurs**.

Pour voir où Podman stocke physiquement le volume : `podman volume inspect host-data` (champ `Mountpoint`). Vous n'avez normalement pas besoin d'y toucher directement.

5.3. Anonymous volume

Il est également possible de demander à Podman de créer un volume **sans lui donner explicitement un nom** :

```
podman run -d --name app -v /data nginx
```

Podman crée alors un volume associé au conteneur. Pour afficher les volumes :

```
podman volume ls
```

Volume sans nom, donc difficile à retrouver

Un volume anonyme reçoit un identifiant aléatoire : il est difficile à reconnaître dans `podman volume ls` et facile à oublier. Pour des données importantes, préférez un **named volume**.

5.4. Résumé

Type	Syntaxe	Source	Géré par
Bind mount	<code>-v /home/student/data:/data</code>	Chemin de l'hôte (existant)	L'administrateur
Named volume	<code>-v acme-data:/data</code>	Nom du volume	Podman
Anonymous volume	<code>-v /data</code>	Aucune (volume sans nom)	Podman

À retenir

- ✓ **Bind mount** : on expose un dossier précis de l'hôte.
- ✓ **Named volume** : Podman gère le stockage, on le retrouve par son nom.
- ✓ **Anonymous volume** : un volume sans nom, créé automatiquement.
- ✓ Pour une base de données, utiliser un volume pour **ne pas perdre les données**.

6. Copier des fichiers avec podman cp

La commande `podman cp` permet de copier des fichiers ou répertoires **Host** → **Container** et **Container** → **Host**.

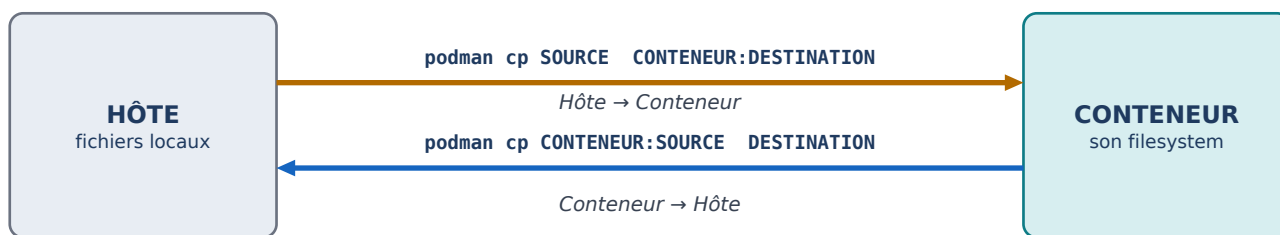


Figure 6 — `podman cp` : la partie contenant « NOM: » désigne toujours le conteneur.

6.1. Host → Container

Syntaxe : `podman cp SOURCE CONTAINER:DESTINATION`. Exemple :

```
podman cp index.html nginx:/usr/share/nginx/html/index.html
```

6.2. Container → Host

Syntaxe : `podman cp CONTAINER:SOURCE DESTINATION`. Exemple :

```
podman cp nginx:/etc/nginx/nginx.conf ./nginx.conf
```

6.3. Copier un répertoire

```
podman cp "$HOME/workspace2/acme-nginx-web/html/." acme-demo-nginx:/usr/share/nginx/html/
```

Le `/.` est utile lorsque l'on souhaite copier **le contenu du répertoire** plutôt que créer un sous-répertoire `html` dans la destination.

6.4. `cp` ou `-v` ?

	<code>podman cp</code>	<code>-v</code> (volume / bind mount)
Nature	Copie ponctuelle (instantané)	Montage permanent
Données	Deux copies indépendantes	Mêmes données visibles des deux côtés
Moment	À tout moment, sur un conteneur existant	Défini à la création du conteneur
Persistance	Perdu si le conteneur est supprimé (sauf copie vers l'hôte)	Conservé après suppression du conteneur

En clair — `cp` vs `-v`

`podman cp`, c'est **envoyer une photocopie** d'un document : après l'envoi, chacun modifie sa copie dans son coin. `-v`, c'est **partager le même classeur** : toute modification est vue immédiatement par les deux.

7. Variables d'environnement

Une **variable d'environnement** permet de transmettre une **configuration** au processus exécuté dans le conteneur. Avec Podman :

```
-e NAME=value
```

Exemple :

```
podman run -d --name app -e RESPONSE="Hello ACME" quay.io/myacme/welcome
```

À l'intérieur du conteneur, `echo "$RESPONSE"` donnera `Hello ACME`. Pour le vérifier depuis l'hôte :

```
podman exec app sh -c 'echo "$RESPONSE"'  
# Hello ACME
```

Guillemets simples ou doubles ?

Écrivez la commande entre **guillemets simples** : avec des guillemets doubles, c'est le shell de **l'hôte** qui remplacerait `$RESPONSE` (variable inconnue, donc vide) avant même d'envoyer la commande au conteneur.

`quay.io/myacme/welcome` est une image d'exemple : `quay.io` est le registry, `myacme` l'organisation, `welcome` le nom de l'image.

8. ARG vs ENV

C'est une **distinction très importante** lors de la construction d'une image.

8.1. ARG

ARG définit une variable utilisée **pendant le build** de l'image. Exemple :

```
FROM docker.io/library/alpine:latest  
  
ARG APP_VERSION  
  
RUN echo "Building version $APP_VERSION"
```

Construction :

```
podman build --build-arg APP_VERSION=1.0 -t acme-app:1.0 .
```

L'argument est fourni au moment du `podman build`.

8.2. ENV

ENV définit une variable d'environnement disponible **dans l'image et lors de l'exécution** du conteneur.

Exemple :

```
FROM docker.io/library/alpine:latest  
  
ENV APP_VERSION=1.0  
  
CMD ["sh", "-c", "echo Version=$APP_VERSION"]
```

Construire, puis lancer :

```
podman build -t acme-app .  
podman run --rm acme-app  
# Version=1.0
```

8.3. Différence entre ARG et ENV

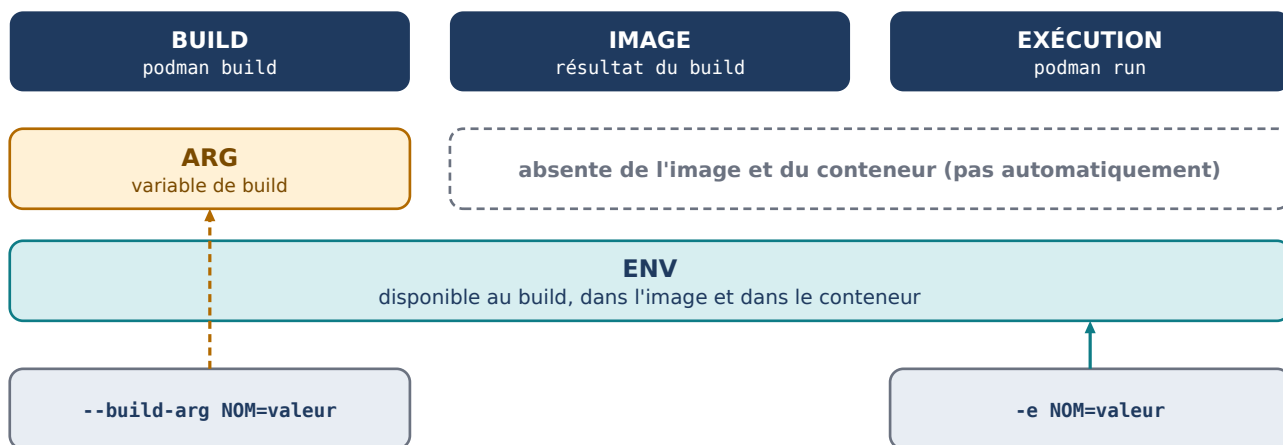


Figure 7 — ARG vit uniquement pendant le build ; ENV est conservé dans l'image et dans le conteneur.

	ARG	ENV
Utilisation principale	Build	Runtime
Fourni avec	--build-arg	-e ou ENV
Disponible pendant le build	Oui	Oui
Disponible dans le conteneur	Pas automatiquement	Oui
Exemple	version de build	configuration de l'application

Pas de secrets dans ARG

Ne pas utiliser ARG comme mécanisme de stockage de **secrets**. Les valeurs utilisées pendant un build peuvent être **exposées dans l'historique ou les métadonnées de l'image**.

En clair — ARG vs ENV

ARG est un **échafaudage** : utile pendant la construction, démonté ensuite. ENV est une **inscription gravée dans le bâtiment** : elle reste visible une fois celui-ci habité (le conteneur en marche).

À retenir

- ✓ ARG = pendant le **build** (--build-arg) ; ENV = dans l'**image** et le **conteneur** (ENV ou -e).
- ✓ -e modifie une variable à l'**exécution** ; --build-arg à la **construction**.

9. Exemple ARG → ENV

Un cas très fréquent consiste à utiliser un **argument de build** pour définir une valeur dans l'**environnement final**.

Containerfile

```
FROM docker.io/library/mariadb:latest

ARG ACME_MARIADB_DATABASE
ARG ACME_MARIADB_PASSWORD

ENV MARIADB_DATABASE=${ACME_MARIADB_DATABASE}
ENV MARIADB_ROOT_PASSWORD=${ACME_MARIADB_PASSWORD}
```

Build :

```
podman build \
  --build-arg ACME_MARIADB_DATABASE=acme \
  --build-arg ACME_MARIADB_PASSWORD=acme \
  -t acme:5000/acme-mariadb:latest .
```

En clair — Nom d'image : registry:port/nom:tag

acme:5000/acme-mariadb:latest se lit : le **registry** acme sur le port 5000, l'image acme-mariadb, le **tag** (version) latest. Ce nom sert à la fois à identifier l'image et à dire où l'envoyer (podman push).

Exemple pédagogique, pas une bonne pratique

Ici le mot de passe est **inscrit dans l'image** via ENV : quiconque peut lire l'image (podman inspect, podman history) peut le voir, ce qui contredit l'avertissement de la section précédente. C'est acceptable pour un exercice, **pas en production** : on y préfère -e au lancement du conteneur ou un mécanisme de secrets (podman secret).

10. Les réseaux Podman

Les applications **multi-conteneurs** ont généralement besoin de communiquer entre elles. On peut créer un réseau :

```
podman network create net
```

Vérifier :

```
podman network ls
```

Pourquoi mettre les conteneurs sur le même réseau ?

Prenons une application : **WordPress** doit contacter **MariaDB**.

```
WordPress
  |
  v
MariaDB
```

Si les deux conteneurs sont connectés au **même réseau Podman**, ils peuvent communiquer via le réseau interne et utiliser le **nom du conteneur comme nom logique**.

```
podman run -d --name mariadb --network net mariadb
podman run -d --name wordpress --network net wordpress
```

L'application WordPress peut alors utiliser mariadb comme **hostname** de la base de données.

En clair — Réseau et nom de conteneur

Un réseau Podman est comme un **réseau d'entreprise avec annuaire** : sur le même réseau, on joint un collègue par son **nom** (mariadb) sans connaître son numéro (adresse IP). Sur des réseaux différents, les conteneurs sont comme deux entreprises sans annuaire commun. Sur le réseau par défaut, la résolution des noms n'est généralement pas activée : c'est pourquoi on crée son propre réseau.

Pourquoi ne pas utiliser localhost ?

Dans un conteneur, localhost désigne **le conteneur lui-même**, pas un autre conteneur. Donc dans WordPress :

```
DB_HOST=localhost    → WordPress → lui-même    (x)
DB_HOST=mariadb      → WordPress → MariaDB      (✓)
```

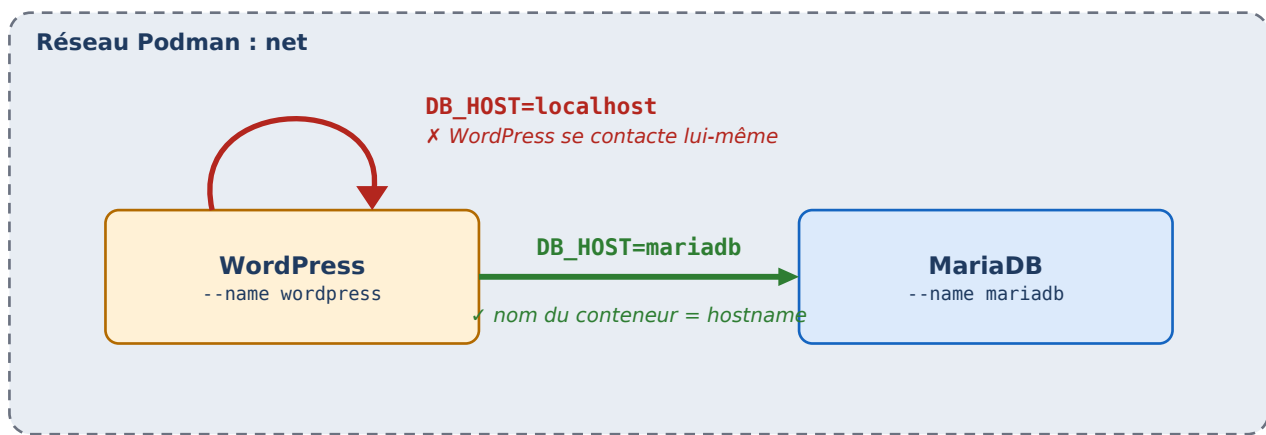


Figure 8 — Même réseau : on joint MariaDB par son nom ; localhost ramène à WordPress lui-même.

Rappel du chapitre sur les namespaces : chaque conteneur possède son **propre namespace réseau**, donc son propre `localhost`.

Exemple complet (illustratif)

Ajout pédagogique : les images officielles exigent quelques variables pour démarrer (mots de passe d'exemple à ne jamais réutiliser en production).

```
podman network create net

podman run -d --name mariadb --network net \
  -e MARIADB_ROOT_PASSWORD=exemple \
  -e MARIADB_DATABASE=wordpress \
  mariadb

podman run -d --name wordpress --network net -p 8080:80 \
  -e WORDPRESS_DB_HOST=mariadb \
  -e WORDPRESS_DB_USER=root \
  -e WORDPRESS_DB_PASSWORD=exemple \
  wordpress
```

À retenir

- ✓ Les conteneurs d'un **même réseau** se joignent par leur **nom**.
- ✓ `localhost` = le conteneur lui-même, **jamais** un autre conteneur.
- ✓ Créer le réseau : `podman network create` ; l'utiliser : `--network`.

11. Autres commandes essentielles à connaître

Conteneurs

Commande	Rôle
<code>podman ps / podman ps -a</code>	Lister les conteneurs en cours / tous
<code>podman run</code>	Créer et démarrer un conteneur à partir d'une image
<code>podman start</code>	Démarrer un conteneur existant arrêté
<code>podman stop</code>	Arrêter un conteneur (il n'est pas supprimé)
<code>podman restart</code>	Arrêter puis redémarrer
<code>podman rm</code>	Supprimer un conteneur
<code>podman inspect</code>	Afficher tous les détails (JSON) d'un conteneur ou d'une image
<code>podman logs</code>	Afficher la sortie du processus (<code>-f</code> pour suivre en direct)
<code>podman exec</code>	Exécuter une commande dans un conteneur en cours (<code>-it ... sh</code> pour y entrer)
<code>podman cp</code>	Copier des fichiers entre hôte et conteneur

Images

Commande	Rôle
<code>podman images</code>	Lister les images locales
<code>podman pull</code>	Télécharger une image depuis un registry
<code>podman build</code>	Construire une image depuis un Containerfile
<code>podman rmi</code>	Supprimer une image
<code>podman inspect</code>	Détails d'une image

Volumes

Commande	Rôle
<code>podman volume create</code>	Créer un volume
<code>podman volume ls</code>	Lister les volumes
<code>podman volume inspect</code>	Détails d'un volume (emplacement, driver...)
<code>podman volume rm</code>	Supprimer un volume

Réseaux

Commande	Rôle
<code>podman network create</code>	Créer un réseau
<code>podman network ls</code>	Lister les réseaux
<code>podman network inspect</code>	Détails d'un réseau (sous-réseau, conteneurs connectés...)
<code>podman network connect</code>	Connecter un conteneur existant à un réseau
<code>podman network disconnect</code>	Déconnecter un conteneur d'un réseau
<code>podman network rm</code>	Supprimer un réseau

Ce qu'il faut absolument retenir

Je veux...	Je fais...
Construire une image	<code>podman build -t nom .</code> (avec un Containerfile)
Lancer en arrière-plan avec un nom	<code>podman run -d --name web image</code>
Accéder à un service depuis l'hôte	<code>-p HOTE:CONTENEUR</code> (ex. <code>-p 8001:80</code>)
Exposer un dossier de l'hôte (avec SELinux)	<code>-v \$HOME/site:/chemin:Z</code>
Partager un volume entre conteneurs (SELinux)	<code>-v \$HOME/data:/data:z</code>
Conserver des données gérées par Podman	<code>podman volume create</code> puis <code>-v nom:/data</code>
Copier un fichier vers / depuis un conteneur	<code>podman cp SRC CTN:DEST</code> / <code>podman cp CTN:SRC DEST</code>
Passer une configuration à l'exécution	<code>-e NOM=valeur</code>
Paramétrer la construction	<code>ARG + --build-arg</code>
Faire communiquer deux conteneurs	<code>podman network create + --network</code> (joindre par le nom)
Voir les logs / entrer dans un conteneur	<code>podman logs nom</code> / <code>podman exec -it nom sh</code>

Erreurs fréquentes

- Inverser `HOTE:CONTENEUR` dans `-p`.
- Croire qu'`EXPOSE` publie le port.
- Utiliser `localhost` pour joindre un autre conteneur.
- Mettre des secrets dans un `ARG` (ou les graver dans l'image avec `ENV`).
- Oublier `:Z` / `:z` sur un système avec SELinux.
- Publier deux conteneurs sur le même port hôte.
- Stocker des données importantes uniquement dans le conteneur (sans volume).

Glossaire des termes importants

Terme	Définition simple
Containerfile	Fichier décrivant comment construire une image (équivalent du Dockerfile).
Image	Modèle en lecture seule, composé de couches, à partir duquel on crée des conteneurs.
Conteneur	Instance en cours d'exécution (ou arrêtée) d'une image.
Registry	Serveur qui stocke et distribue des images (docker.io, quay.io...).
Tag	Étiquette de version d'une image (<code>latest</code> , <code>1.0...</code>).
Port mapping	Redirection d'un port de l'hôte vers un port du conteneur (<code>-p</code>).
Détaché (-d)	Conteneur exécuté en arrière-plan.
Volume	Espace de stockage dont la durée de vie est indépendante de celle du conteneur.
Bind mount	Montage d'un chemin existant de l'hôte dans le conteneur.
Named volume	Volume nommé, dont le stockage est géré par Podman.
Anonymous volume	Volume créé sans nom explicite.
SELinux	Système de contrôle d'accès supplémentaire basé sur des étiquettes (contextes).
Relabel (:Z, :z)	Réétiquetage SELinux du volume : <code>:Z</code> privé (un conteneur), <code>:z</code> partagé.
Variable d'environnement	Paire NOM=valeur transmise au processus pour le configurer.
ARG	Variable disponible uniquement pendant le build.
ENV	Variable conservée dans l'image et disponible dans le conteneur.
Réseau Podman	Réseau virtuel reliant des conteneurs ; ils s'y joignent par leur nom.
Hostname	Nom d'une machine sur le réseau ; ici, le nom du conteneur.
<code>localhost</code>	Désigne toujours la machine/le conteneur courant.
Éphémère	Qui disparaît avec le conteneur : le filesystem interne du conteneur est éphémère.
Multi-conteneurs	Application composée de plusieurs conteneurs qui coopèrent (ex. WordPress + MariaDB).

Questions de compréhension — avec corrigés

Essayez de répondre de vous-même avant de lire la correction.

Question 1 Quelle est la différence entre un port du conteneur et un port de l'hôte ?

Correction

Le **port du conteneur** est celui sur lequel l'application écoute **à l'intérieur** du conteneur (nginx : 80), dans son réseau isolé.

Le **port de l'hôte** est celui de la machine, par lequel on accède au service depuis l'extérieur. Le **port mapping** (-p) relie les deux.

Question 2 Que signifie -p 8001:80 ?

Correction

Le port **8001 de l'hôte** est redirigé vers le port **80 du conteneur**. Ordre : `HOST_PORT:CONTAINER_PORT`. On accède alors à nginx avec `curl http://localhost:8001`.

Question 3 Pourquoi EXPOSE 80 ne suffit-il pas pour accéder à Nginx depuis l'hôte ?

Correction

`EXPOSE` ne fait que **documenter** le port utilisé par l'application ; il ne le **publie pas**. Il faut utiliser `-p` (par exemple `-p 8080:80`) au lancement du conteneur. (L'option `-P` publie tous les ports exposés, mais sur des ports hôtes aléatoires.)

Question 4 Quelle est la différence entre un bind mount et un named volume ?

Correction

Un **bind mount** monte un **chemin existant de l'hôte** (`-v /home/student/data:/data`) : l'administrateur le gère directement.

Un **named volume** est un volume **géré par Podman**, identifié par son **nom** (`-v acme-data:/data`) : Podman s'occupe du stockage physique.

Question 5 Pourquoi utiliser un volume pour une base de données ?

Correction

Parce que le filesystem du conteneur est **éphémère** : sans volume, les données sont **perdues** quand le conteneur est supprimé. Un volume les conserve **indépendamment du cycle de vie** du conteneur, permet de remplacer ou mettre à jour le conteneur sans perte et facilite les sauvegardes.

Question 6 Quelle est la différence entre `podman cp` et `-v` ?

Correction

`podman cp` fait une **copie ponctuelle** : les deux copies deviennent indépendantes. `-v` crée un **montage permanent** défini à la création du conteneur : les mêmes données sont visibles des deux côtés, en temps réel, et persistent après suppression du conteneur.

Question 7 Dans quel sens fonctionne `podman cp` ?

Correction

Dans **les deux sens**. Host → Container : `podman cp SOURCE CONTAINER:DESTINATION`. Container → Host : `podman cp CONTAINER:SOURCE DESTINATION`. Le côté qui contient `NOM_DU_CONTENEUR:` est celui du conteneur.

Question 8 Quelle est la différence entre ARG et ENV ?**Correction**

ARG est une variable utilisée **pendant le build** (fournie avec `--build-arg`), non disponible automatiquement dans le conteneur.

ENV est une variable **d'environnement** disponible pendant le build, dans l'image et **dans le conteneur** (fournie par `ENV` ou `-e`).

Question 9 Quand utilise-t-on --build-arg ?**Correction**

Au moment de `podman build`, pour **paramétrer la construction** de l'image (version à installer, nom...), à condition que le Containerfile déclare l'ARG correspondant.

Question 10 Quand utilise-t-on -e ?**Correction**

Au moment de `podman run`, pour **configurer l'application à l'exécution** (message, adresse de base de données...) sans reconstruire l'image.

Question 11 Pourquoi ne faut-il pas utiliser ARG pour stocker des secrets ?**Correction**

Parce que les valeurs utilisées pendant un build peuvent être **exposées dans l'historique ou les métadonnées de l'image** : toute personne pouvant lire l'image pourrait les récupérer.

Question 12 Pourquoi deux conteneurs ne peuvent-ils pas publier simultanément le même port de l'hôte ?**Correction**

Parce qu'un port de l'hôte ne peut être **écouté que par un seul processus à la fois**. Le second conteneur échouerait (port déjà utilisé). Solution : publier sur des ports hôtes différents (`-p 8001:80` et `-p 8002:80`). À l'intérieur, les deux peuvent écouter sur le port 80 car leurs réseaux sont isolés.

Question 13 Pourquoi WordPress et MariaDB doivent-ils être sur le même réseau ?**Correction**

Pour qu'ils puissent **communiquer via le réseau interne** et que WordPress joigne la base en utilisant le **nom du conteneur** (`mariadb`) comme hostname.

Question 14 Pourquoi localhost ne permet-il pas à WordPress de joindre MariaDB ?**Correction**

Dans un conteneur, `localhost` désigne **le conteneur lui-même** (chaque conteneur a son propre namespace réseau). `DB_HOST=localhost` ferait donc contacter WordPress à lui-même ; il faut `DB_HOST=mariadb`.

Question 15 Quel est le rôle de podman network inspect ?**Correction**

Il affiche les **détails d'un réseau** : nom, driver, sous-réseau, passerelle, résolution DNS, conteneurs connectés... Utile pour diagnostiquer un problème de communication.

Question 16 Quel est le rôle de `podman volume inspect` ?**Correction**

Il affiche les **détails d'un volume** : nom, driver, date de création et surtout son **emplacement sur l'hôte** (Mountpoint).

Question 17 Comment vérifier les logs d'un conteneur ?**Correction**

Avec `podman logs NOM` (option `-f` pour suivre en direct, `--tail N` pour les N dernières lignes).

Question 18 Comment entrer dans un conteneur en cours d'exécution ?**Correction**

Avec `podman exec -it NOM sh` (ou `bash` si disponible) : `-i` garde l'entrée ouverte, `-t` fournit un terminal.

Question 19 Quelle est la différence entre `podman stop` et `podman rm` ?**Correction**

`podman stop` **arrête** le processus du conteneur mais le conteneur **existe encore** (visible avec `podman ps -a`, redémarrable avec `podman start`).

`podman rm` **supprime** le conteneur (qui doit être arrêté, ou `-f`). Les données de son filesystem interne sont perdues ; les volumes nommés, eux, restent.

Question 20 Quelle est la différence entre une image et un conteneur ?**Correction**

Une **image** est un **modèle en lecture seule** (créé par `podman build` ou `podman pull`). Un **conteneur** est une **instance** créée à partir de cette image (`podman run`), avec sa propre couche modifiable. Une image peut servir à lancer plusieurs conteneurs.